

Университет ИТМО

Расширенный курс по backend-разработке в среде Node.js

Домашняя работа №5

Реализация межсервисного взаимодействия посредством RabbitMQ

Платформа для фитнес-тренировок и здоровья

Выполнила: Яковенко Ксения

Группа: БР1.1

2026 г.

1. Цель и постановка задачи

Цель работы - подключить RabbitMQ к микросервисной версии фитнес-платформы и заменить одно синхронное межсервисное действие асинхронной передачей события через очередь сообщений.

По требованиям Д35 необходимо настроить RabbitMQ или Kafka и реализовать межсервисное взаимодействие с помощью выбранного брокера сообщений. Для реализации выбран RabbitMQ и сценарий создания уведомления после записи пользователя на тренировочный план.

Компонент	Роль
Training Plan Service	Создаёт запись пользователя на план и публикует событие.
RabbitMQ	Маршрутизирует и хранит сообщение до подтверждения consumer-a.
Notification Service	Получает событие и создаёт уведомление.
plans_db	Хранит user_training_plans и outbox_events.
notification_db	Хранит уведомления и eventId обработанного события.

2. Выбранный событийный сценарий

В ЛР2 Training Plan Service напрямую вызывал внутренний REST endpoint Notification Service. В Д35 этот вызов заменён событием training-plan.enrolled.v1. Основная операция записи на план не зависит от времени обработки уведомления, поэтому сценарий хорошо подходит для асинхронного взаимодействия.

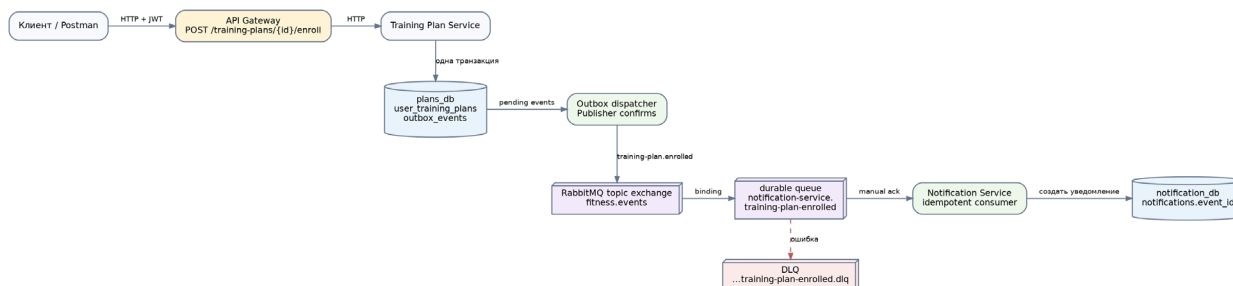


Рисунок 1 - Передача события о записи на тренировочный план

3. Топология RabbitMQ

Элемент	Значение	Назначение
Exchange	fitness.events	Durable topic exchange для интеграционных событий.
Routing key	training-plan.enrolled	Маршрутизация события записи на план.
Queue	notification-service.training-plan-enrolled	Очередь consumer-a Notification Service.
DLX	fitness.events.dlx	Exchange для сообщений, обработка которых завершилась ошибкой.
DLQ	notification-service.training-plan-enrolled.dlq	Хранение ошибочных сообщений для анализа.

Exchange и очереди объявляются с параметром durable, сообщения публикуются с persistent=true. Consumer использует ручное подтверждение: ack отправляется только после успешной записи уведомления в базу.

4. Формат интеграционного события

```
{
  "id": "4f4935b8-...",
  "type": "training-plan.enrolled.v1",
  "version": 1,
  "occurredAt": "2026-06-29T12:00:00.000Z",
  "payload": {
    "userId": 2,
    "planId": 1,
    "userTrainingPlanId": 1,
    "planTitle": "Старт за одну неделю",
    "startDate": "2026-06-29"
  }
}
```

Поле id используется не только для трассировки, но и для защиты от повторной обработки. Версия события включена в type и отдельное поле version, что позволяет развивать контракт без незаметного нарушения consumer-ов.

5. Transactional Outbox в Training Plan Service

Простая публикация после INSERT может привести к потере события: запись в БД уже сохранена, а RabbitMQ в этот момент недоступен. Поэтому добавлена таблица outbox_events. Пользовательская запись и outbox-событие сохраняются одной транзакцией.

```
await queryRunner.startTransaction();
userTrainingPlan = await userPlanRepository.save(...);
await outboxRepository.save({
  id: eventId,
  eventType: 'training-plan.enrolled.v1',
  routingKey: 'training-plan.enrolled',
  payload: { userId, planId, userTrainingPlanId, planTitle, startDate }
});
await queryRunner.commitTransaction();
```

Фоновый dispatcher периодически выбирает записи, у которых publishedAt IS NULL. После publisher confirm поле publishedAt заполняется. При ошибке увеличивается attempts и сохраняется lastError; событие остаётся доступным для повторной попытки.

6. Publisher и consumer

Механизм	Реализация
Переподключение	Publisher и consumer повторяют подключение после потери RabbitMQ.
Publisher confirms	Outbox помечается опубликованным только после waitForConfirms().
Manual ack	Сообщение подтверждается после успешной транзакции Notification Service.
Idempotency	notifications.eventId имеет unique constraint; повторная доставка игнорируется.
Dead-letter	При исключении выполняется nack(requeue=false), сообщение уходит в DLQ.
Prefetch	Consumer ограничивает число одновременно выданных сообщений.

Семантика доставки: RabbitMQ и outbox обеспечивают доставку «как минимум один раз». Поэтому идемпотентность consumer-а обязательна.

7. Конфигурация и запуск

Для локальной разработки PostgreSQL и RabbitMQ запускаются через docker-compose.infrastructure.yml, а Node.js-сервисы работают на хосте.

```
Copy-Item .env.example .env
npm.cmd install
npm.cmd run infra:up
npm.cmd run build
npm.cmd run seed
npm.cmd run dev
```

RabbitMQ Management доступен по адресу <http://localhost:15672> с учётными данными `fitness / fitness`. В интерфейсе можно показать exchange, очередь, binding, количество сообщений и DLQ.

8. Порядок проверки

1. Открыть <http://localhost:3000/health/services> и убедиться, что все сервисы отвечают ok.
2. Выполнить Login user в Postman и получить JWT.
3. Выполнить Enroll user. Ответ должен содержать `eventQueued=true` и `eventId`.
4. В RabbitMQ Management открыть очередь `notification-service.training-plan-enrolled`.
5. Через 1-3 секунды выполнить запрос Notifications и проверить созданное уведомление.
6. Сравнить `eventId` в ответе enrollment и поле `eventId` уведомления.

```
GET http://localhost:3003/health/rabbitmq
GET http://localhost:3007/health/rabbitmq
```

9. Обработка ошибок

Ситуация	Поведение системы
RabbitMQ временно недоступен	Enrollment и outbox сохраняются; dispatcher повторяет публикацию.
Соединение consumer-а разорвано	Consumer переподключается и продолжает чтение durable queue.
Сообщение доставлено повторно	Повторный eventId найден, новое уведомление не создаётся.
Ошибка payload/БД consumer-а	Сообщение направляется в DLQ для анализа.
Процесс publisher-а остановлен	Неопубликованные записи остаются в outbox_events и будут обработаны после запуска.

10. Результат

В результате синхронная зависимость Training Plan Service от Notification Service заменена интеграционным событием RabbitMQ. Реализация содержит durable topology, publisher confirms, manual ack, DLQ, transactional outbox и идиempотентный consumer. TypeScript-код прошёл статическую компиляцию, а JSON/YAML-файлы - синтаксическую проверку.